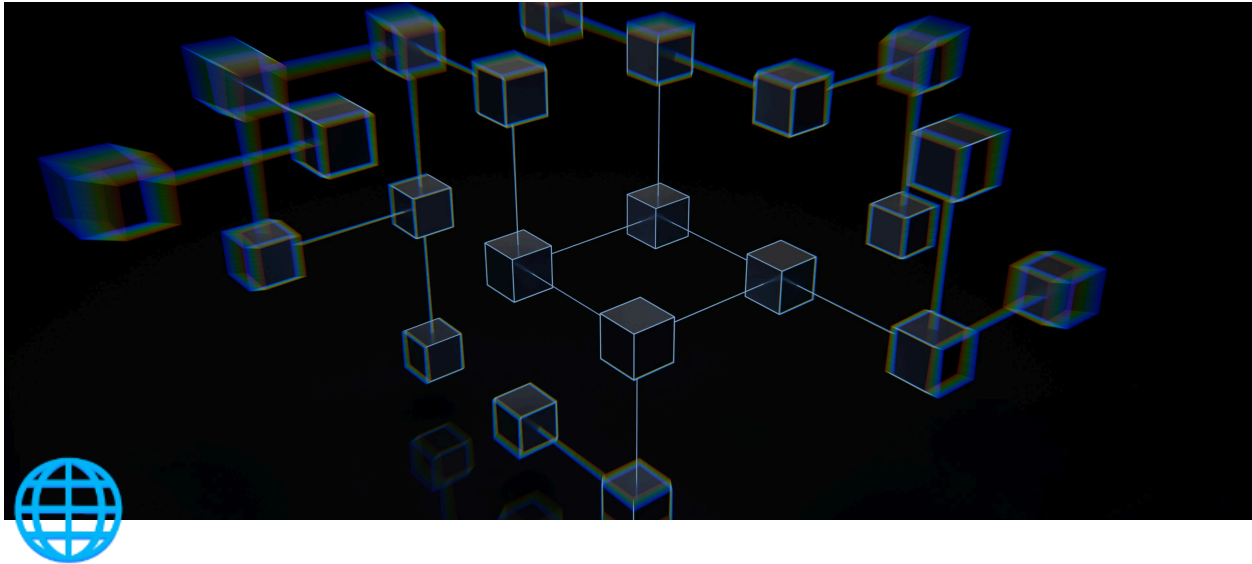




Network Monitoring & Predictive Failure System

1 Project Overview

The **Network Monitoring & Predictive Failure System** is a real-time desktop-based network monitoring application built using Python and PySide6 (Qt).

The system continuously monitors multiple network nodes using ICMP (ping), evaluates their health based on latency and failure count, logs structured data to MongoDB and local files, and provides a real-time graphical dashboard with alerts and analytics.

The architecture is modular and supports future integration of a RAG-based predictive failure model.

2 Core Objectives

- Monitor network nodes in real time

- Detect connectivity failures
 - Measure latency
 - Classify node state (UP / DEGRADED / DOWN)
 - Store structured logs in MongoDB
 - Maintain backup logs in log.txt
 - Provide authentication (Login / Signup)
 - Provide time-based log filtering
 - Display real-time dashboard visualization
 - Support future predictive analytics
-

Technology Stack

Programming Language

- Python 3.x

GUI Framework

- PySide6 (Qt for Python)

Database

- MongoDB Atlas
- PyMongo

System Tools

- subprocess (ICMP ping)
 - Multithreading
 - QTimer (GUI refresh loop)
-

4 System Architecture

gui

- |— __init__.py
- |— alerts.py
- |— chart.py
- |— data.py
- |— global_chart.py
- |— gui.py
- |— login_window.py
- |— node_availability_chart.py
- |— node_pie_chart.py
- |— signup_window.py
- |— summary.py
- |— widgets.py

logs

- |— log.txt

- |— db
 - |— __init__.py
 - |— file_log_service.py
 - |— log_service.py
 - |— mongo.py
 - |— user_service.py
- |— config.py
- |— main.py
- |— monitor.py
- |— ping.py
- |— run_gui.py
- |— run_monitor.py

The system is divided into:

- Monitoring Engine
 - Database Services
 - GUI Layer
 - Logging Layer
 - Authentication Layer
-

Monitoring Engine

File: `monitor.py`

The monitoring engine:

1. Iterates over configured nodes
 2. Sends ICMP ping using subprocess
 3. Evaluates status
 4. Updates failure count
 5. Stores log
 6. Generates alerts if state changes
-

5.1 Ping Logic

File: `ping.py`

```
subprocess.run(["ping", "-c", "1", ip])
```

Extracts:

- success (returncode)
- latency (parsed from "time=")

Returns:

```
{
    success: True/False,
    latency: float or None,
    error: None or error_type
}
```

5.2 Status Evaluation Logic

Rules:

Condition	State
success = True AND latency <= threshold	UP
success = True AND latency > threshold	DEGRADED
success = False (3+ consecutive fails)	DOWN

Fail count increases when ping fails.

6 Node Model

Each node contains:

```
{
    node_name: str,
    ip: str,
    network_type: str,
    state: str,
    last_latency: float,
    fail_count: int
}
```

7 Logging System

The system uses **dual logging**:

7.1 MongoDB Logging

Collection: `logs`

Document structure:

```
{
  timestamp: datetime,
  node_name: str,
  ip: str,
  network_type: str,
  state: str,
  latency: float,
  fails: int
}
```

Used for:

- Filtering
- Analytics
- Future ML
- Alert generation

7.2 File Logging (Backup)

File: `logs/log.txt`

Format:

```
2026-02-12 14:40:42 | node_2_laptop | 192.168.1.33 | UP | typ
e = wifi | latency=0.265 | fails=0
```

Ensures:

- Offline resilience

- Debug trace
 - Backup logging
-

8 Alert System

File: `alert_service.py`

Alerts are generated when:

```
node["state"] != previous_state
```

Severity Mapping:

State	Severity
UP (Recovery)	INFO
DEGRADED	WARNING
DOWN	CRITICAL

Stored in MongoDB:

```
{
  timestamp,
  node_name,
  ip,
  state,
  severity
}
```

Displayed in GUI in color-coded format.

9 Authentication System

Files:

- `login_window.py`

- signup_window.py
- user_service.py

MongoDB Collection: `users`

User document:

```
{  
  email: str,  
  password: str  
}
```

Authentication flow:

```
Login Window  
  ↓  
MongoDB find_one(email, password)  
  ↓  
If valid → Open Dashboard
```

10 GUI System

Main file: `gui.py`

Built using PySide6.

Dashboard includes:

- Node list (left sidebar)
- Node detail panel
- Latency chart
- Global latency chart
- Availability pie chart
- Alert panel
- Log filter dropdown

1.1 Log Filtering System

Dropdown above Alerts panel:

Options:

- All
- Last 24 Hours
- Last 7 Days
- Last 30 Days

MongoDB Query:

```
{
  timestamp: { "$gte": datetime.now() - timedelta(days=X) }
}
```

This filters logs in real-time.

1.2 Multithreading Architecture

Main thread:

- Runs GUI

Background thread:

- Runs monitor_loop()

Why?

Qt requires single GUI thread.

Without threading → GUI freezes.

1.3 Handling DHCP Dynamic IP Issue

Problem observed:

Laptop IP changed from:

192.168.1.33 → 192.168.1.34

Cause:

Router DHCP assigns dynamic IP.

Solution:

- Use static IP
 - OR
 - Monitor router instead
 - OR
 - Use hostname.local
-

14 Current Features Implemented

- ✓ Real-time monitoring
 - ✓ Latency tracking
 - ✓ Failure detection
 - ✓ Alert severity system
 - ✓ MongoDB logging
 - ✓ File logging
 - ✓ Login/Signup authentication
 - ✓ Log filtering
 - ✓ Multithreaded architecture
 - ✓ GUI visualization
-

15 Limitations

- Plain-text password storage (not hashed yet)

- ICMP-based only (no SNMP)
 - Local desktop only (not web-based)
 - No distributed monitoring agents
 - No predictive ML yet
-

16 Future Enhancements

RAG-based Predictive Model

Planned architecture:

```
Logs → MongoDB
      ↓
Embedding Model
      ↓
Vector Database (FAISS)
      ↓
Pattern Similarity
      ↓
Predict Failure Probability
```

Goal:

Predict node failure before it happens.

Additional Improvements

- Password hashing (bcrypt)
- Role-based authentication
- REST API layer
- Cloud deployment
- Docker containerization

- Web dashboard (React frontend)
 - SNMP integration
 - Email/SMS alerting
-

17 How To Run Project

From root folder:

```
python src/main.py
```

Requirements:

```
pip install pymongo dnspython PySide6 python-dotenv
```

MongoDB URI stored in `.env`

18 Summary

This project demonstrates:

- Real-time systems engineering
- Network monitoring fundamentals
- Multithreading
- GUI application development
- Database integration
- Modular architecture
- Alerting systems
- Log analytics

It is a complete system-level application combining:

Backend + System Programming + Database + GUI + Authentication.

code:

gui/alerts.py

```
from PySide6.QtWidgets import QWidget, QVBoxLayout, QLabel, Q
ListWidget, QListWidgetItem
from PySide6.QtGui import QColor
from datetime import datetime
from db.log_service import get_logs_by_range

class AlertsPanel(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)

        layout = QVBoxLayout()
        self.setLayout(layout)

        title = QLabel("Alerts")
        title.setStyleSheet("font-size: 16px; font-weight: bo
ld;")

        self.counter_label = QLabel("Total: 0")
        self.counter_label.setStyleSheet("font-size: 12px; co
lor: #888;")

        self.list_widget = QListWidget()

        layout.addWidget(title)
        layout.addWidget(self.counter_label)
        layout.addWidget(self.list_widget)

        self.previous_states = {}
```

```

# LIMIT alerts
self.max_alerts = 100

def update_alerts(self, states):
    current_time = datetime.now().strftime("%H:%M:%S")

    for node, info in states.items():
        current = info["state"]
        previous = self.previous_states.get(node)

        if previous and previous != current:
            if current == "DOWN":
                severity = "CRITICAL"
                color = QColor("#e74c3c")
                status = "DOWN"

            elif current == "DEGRADED":
                severity = "WARNING"
                color = QColor("#f1c40f")
                status = "DEGRADED"

            elif current == "UP":
                severity = "INFO"
                color = QColor("#2ecc71")
                status = "RECOVERED"
            else:
                return

        message = f"{current_time} [{severity}] {no
de} {status}"

        item = QListWidgetItem(message)
        font = item.font()
        if severity == "CRITICAL":
            font.setBold(True)
        item.setFont(font)

```

```

        item.setForeground(color)

        # insert at top
        self.list_widget.insertItem(0, item)

        # Remove old alerts if exceeding limit
        if self.list_widget.count() > self.max_alert
s:
            self.list_widget.takeItem(self.list_widge
t.count() - 1)

        # Update counter
        self.counter_label.setText(f"Total: {self.lis
t_widget.count()}")

        self.previous_states[node] = current

def update_alerts_filtered(self, range_type="all"):
    logs = get_logs_by_range(range_type)
    # populate the internal QListWidget
    self.list_widget.clear()

    for log in logs:
        state = log.get("state")

        if state == "DOWN":
            severity = "CRITICAL"
            color = QColor("#e74c3c")
            status = "DOWN"
        elif state == "DEGRADED":
            severity = "WARNING"
            color = QColor("#f1c40f")
            status = "DEGRADED"
        elif state == "UP":
            severity = "INFO"

```

```

        color = QColor("#2ecc71")
        status = "RECOVERED"
    else:
        severity = "INFO"
        color = QColor("#888")
        status = state or "UNKNOWN"

    timestamp = log.get("timestamp")
    try:
        timestamp_str = timestamp.strftime("%H:%M:%S")
    except Exception:
        timestamp_str = str(timestamp)

    message = (
        f"{timestamp_str} [{severity}] {log.get('node_name')}"
        f"{status}"
    )

    item = QListWidgetItem(message)
    font = item.font()
    if severity == "CRITICAL":
        font.setBold(True)
    item.setFont(font)
    item.setForeground(color)

    self.list_widget.addItem(item)

    # Update counter
    self.counter_label.setText(f"Total: {self.list_widget.count()}")

```

gui/charts.py

```
# gui/chart.py
```



```

from PySide6.QtCharts import QChart, QChartView, QLineSeries,
QValueAxis
from PySide6.QtCore import Qt
from PySide6.QtGui import QPainter

class LatencyChart(QChartView):
    def __init__(self, parent=None):
        super().__init__(parent)

        self.chart_obj = QChart()
        self.chart_obj.setTitle("Latency History")

        self.series = QLineSeries()

        # Add series
        self.chart_obj.addSeries(self.series)

        # Axes
        self.axisX = QValueAxis()
        self.axisX.setTitleText("Samples")

        self.axisY = QValueAxis()
        self.axisY.setTitleText("Latency (ms)")

        # Add axes to chart
        self.chart_obj.addAxis(self.axisX, Qt.AlignBottom)
        self.chart_obj.addAxis(self.axisY, Qt.AlignLeft)

        # Attach series to axes
        self.series.attachAxis(self.axisX)
        self.series.attachAxis(self.axisY)

        # Set chart
        self.setChart(self.chart_obj)

```

```

# Enable smooth rendering
self.setRenderHint(QPainter.Antialiasing)

def update_data(self, latency_list):

    # Clear old points
    self.series.clear()

    if not latency_list:
        return

    # Add new points
    for i, latency in enumerate(latency_list):
        self.series.append(i, latency)

    # Calculate axis range
    min_y = min(latency_list)
    max_y = max(latency_list)

    padding = 10

    self.axisX.setRange(0, len(latency_list))
    self.axisY.setRange(max(0, min_y - padding), max_y +
padding)

    # THIS IS THE IMPORTANT FIX – FORCE REDRAW
    self.chart_obj.update()
    self.update()

```

gui/data.py

```

# gui/data.py
import os

LOG_FILE = os.path.join(os.path.dirname(os.path.dirname(__fil
e__)), "logs", "log.txt")

```

```

def parse_key_value(text):
    if "=" not in text:
        return text.strip()

    return text.split("=", 1)[1].strip()

# Latest state per node
def read_latest_node_states():

    states = {}

    if not os.path.exists(LOG_FILE):
        return states

    with open(LOG_FILE, "r") as f:
        for line in f:
            parts = [p.strip() for p in line.split("|")]

            if len(parts) != 7:
                continue

            # CORRECT ORDER: timestamp | node | ip | state |
            type | latency | fails
            timestamp = parts[0]
            node = parts[1]
            ip = parts[2]
            state = parts[3]
            network_type = parse_key_value(parts[4])
            latency = parse_key_value(parts[5])
            fails = parse_key_value(parts[6])

            states[node] = {
                "ip": ip,

```

```

        "network_type": network_type,
        "state": state,
        "latency": latency,
        "fails": fails,
    }

    return states

# Latency history per node
def get_node_latency_history(node_name):

    history = []

    if not os.path.exists(LOG_FILE):
        return history

    with open(LOG_FILE, "r") as f:
        for line in f:
            parts = [p.strip() for p in line.split("|")]

            if len(parts) != 7:
                continue

            node = parts[1]

            if node != node_name:
                continue

            latency = parse_key_value(parts[5])

            try:
                history.append(float(latency))
            except:
                pass

```

```

    return history[-50:]

# Global latency history
def get_global_latency_history():

    timestamp_map = {}

    if not os.path.exists(LOG_FILE):
        return []

    with open(LOG_FILE, "r") as f:
        for line in f:
            parts = [p.strip() for p in line.split("|")]

            if len(parts) != 7:
                continue

            timestamp = parts[0]
            latency = parse_key_value(parts[5])

            try:
                latency = float(latency)
            except:
                continue

            timestamp_map.setdefault(timestamp, []).append(latency)

    history = []

    for t in sorted(timestamp_map.keys()):
        values = timestamp_map[t]
        history.append(sum(values) / len(values))

```

```

return history[-50:]

# State distribution per node
def get_node_state_distribution(node_name):

    up = 0
    degraded = 0
    down = 0

    if not os.path.exists(LOG_FILE):
        return up, degraded, down

    with open(LOG_FILE, "r") as f:
        for line in f:
            parts = [p.strip() for p in line.split("|")]

            if len(parts) != 7:
                continue

            node = parts[1]
            state = parts[3]

            if node != node_name:
                continue

            if state == "UP":
                up += 1

            elif state == "DEGRADED":
                degraded += 1

            elif state == "DOWN":
                down += 1

```

```
return up, degraded, down
```

gui/global_chart.py

```
from PySide6.QtCharts import QChart, QChartView, QLineSeries,
QValueAxis
from PySide6.QtCore import Qt
from PySide6.QtGui import QPainter

class GlobalLatencyChart(QChartView):
    def __init__(self, parent=None):
        super().__init__(parent)

        self.chart_obj = QChart()
        self.chart_obj.setTitle("Global Average Latency")

        self.series = QLineSeries()
        self.chart_obj.addSeries(self.series)

        self.axisX = QValueAxis()
        self.axisX.setTitleText("Time")

        self.axisY = QValueAxis()
        self.axisY.setTitleText("Avg Latency (ms)")

        self.chart_obj.addAxis(self.axisX, Qt.AlignBottom)
        self.chart_obj.addAxis(self.axisY, Qt.AlignLeft)

        self.series.attachAxis(self.axisX)
        self.series.attachAxis(self.axisY)

        self.setChart(self.chart_obj)
        self.setRenderHint(QPainter.Antialiasing)
```

```

def update_data(self, latency_list):

    self.series.clear()

    if not latency_list:
        return

    for i, latency in enumerate(latency_list):
        self.series.append(i, latency)

    min_y = min(latency_list)
    max_y = max(latency_list)

    padding = 10

    self.axisX.setRange(0, len(latency_list))
    self.axisY.setRange(max(0, min_y - padding), max_y +
padding)

    self.chart_obj.update()
    self.update()

```

gui/gui.py

```

# gui/gui.py

import sys
import os

# Add gui directory to path
sys.path.insert(0, os.path.dirname(os.path.abspath(__file_
_)))

from PySide6.QtWidgets import (
    QApplication,
    QMainWindow,

```



```

        QLabel,
        QWidget,
        QHBoxLayout,
        QVBoxLayout,
        QComboBox
    )

from PySide6.QtCore import QTimer

# charts
from chart import LatencyChart
from global_chart import GlobalLatencyChart
from node_availability_chart import NodeAvailabilityChart

# data
from data import (
    read_latest_node_states,
    get_node_latency_history,
    get_global_latency_history,
    get_node_state_distribution,
)

# widgets
from widgets import NodeListWidget
from summary import HealthSummaryWidget
from alerts import AlertsPanel

class NetworkMonitorGUI(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Network Health Monitor")
        self.setGeometry(100, 100, 900, 600)

        self.selected_node = None

```

```

self.states = {}

# =====
# Central widget
# =====

central_widget = QWidget()
self.setCentralWidget(central_widget)

main_layout = QVBoxLayout()
central_widget.setLayout(main_layout)

# =====
# Summary widget
# =====

self.summary = HealthSummaryWidget()
main_layout.addWidget(self.summary)

# =====
# Global latency chart
# =====

self.global_chart = GlobalLatencyChart()
main_layout.addWidget(self.global_chart)

# =====
# Content layout
# =====

content_layout = QHBoxLayout()
main_layout.addLayout(content_layout)

# LEFT SIDE
left_layout = QHBoxLayout()
content_layout.addLayout(left_layout, 3)

```

```

# RIGHT SIDE (alerts)
# Alerts layout with filter
alerts_layout = QVBoxLayout()

self.filter_dropdown = QComboBox()

self.filter_dropdown.addItem([
    "All",
    "Last 24 Hours",
    "Last 7 Days",
    "Last 30 Days"
])

self.filter_dropdown.currentIndexChanged.connect(self.filter_changed)

alerts_layout.addWidget(self.filter_dropdown)

self.alerts_panel = AlertsPanel()

alerts_layout.addWidget(self.alerts_panel)

content_layout.addLayout(alerts_layout, 1)

# =====
# Sidebar
# =====

self.node_list = NodeListWidget(self)
self.node_list.itemClicked.connect(self.on_node_clicked)

left_layout.addWidget(self.node_list)

```

```

# =====
# Details section
# =====

right_layout = QVBoxLayout()

# Node detail label
self.detail_label = QLabel("Select a node to view det
ails")
self.detail_label.setStyleSheet("font-size: 16px;")

right_layout.addWidget(self.detail_label)

# Latency chart
self.chart = LatencyChart()
right_layout.addWidget(self.chart)

# =====
# NODE AVAILABILITY PIE CHART (FIX ADDED)
# =====

self.node_availability_chart = NodeAvailabilityChart
()
self.node_availability_chart.setMinimumHeight(250)

right_layout.addWidget(self.node_availability_chart)

# add details section to layout
left_layout.addLayout(right_layout)

# =====
# Timer
# =====

self.timer = QTimer()
self.timer.timeout.connect(self.refresh_nodes)

```

```

        self.timer.start(2000)

        # Initial refresh
        self.refresh_nodes()

# =====
# Node clicked
# =====

def on_node_clicked(self, item):

    node = item.text().split()[1]

    self.selected_node = node

    info = self.states.get(node)

    if not info:
        return

    # update details text
    details = (
        f"Node: {node}\n"
        f"IP: {info['ip']}\n"
        f"Network: {info['network_type']}\n"
        f"State: {info['state']}\n"
        f"Latency: {info['latency']} ms\n"
        f"Fails: {info['fails']}"
    )

    self.detail_label.setText(details)

    # update latency chart
    latency_history = get_node_latency_history(node)
    self.chart.update_data(latency_history)

```

```

# =====
# UPDATE PIE CHART (FIX)
# =====

up, degraded, down = get_node_state_distribution(nod
e)

self.node_availability_chart.update_data(up, degrade
d, down)

# =====
# Refresh GUI
# =====

def refresh_nodes(self):

    print("Refreshing GUI...")

    # read states
    self.states = read_latest_node_states()

    # update sidebar
    self.node_list.refresh(self.states, self)

    # update summary
    self.summary.update_summary(self.states)

    # update alerts
    # self.alerts_panel.update_alerts(self.states)
    self.filter_changed()

    # update global chart
    global_history = get_global_latency_history()
    self.global_chart.update_data(global_history)

    # update selected node charts

```

```

        if self.selected_node:
            info = self.states.get(self.selected_node)

            if not info:
                return

            # update latency chart
            latency_history = get_node_latency_history(self.selected_node)

            self.chart.update_data(latency_history)

            # update pie chart (FIX)
            up, degraded, down = get_node_state_distribution(
                self.selected_node)

            self.node_availability_chart.update_data(up, degraded, down)

            # update text
            details = (
                f"Node: {self.selected_node}\n"
                f"IP: {info['ip']}\n"
                f"Network: {info['network_type']}\n"
                f"State: {info['state']}\n"
                f"Latency: {info['latency']} ms\n"
                f"Fails: {info['fails']}"
            )

            self.detail_label.setText(details)

# =====
# Filter changed button
# =====
def filter_changed(self):

```

```

        text = self.filter_dropdown.currentText()

        if text == "All":
            self.alerts_panel.update_alerts_filtered("all")

        elif text == "Last 24 Hours":
            self.alerts_panel.update_alerts_filtered("day")

        elif text == "Last 7 Days":
            self.alerts_panel.update_alerts_filtered("week")

        elif text == "Last 30 Days":
            self.alerts_panel.update_alerts_filtered("month")

```

gui/login_window.py

```

import sys
import os

# Add parent directory to path
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from PySide6.QtWidgets import (
    QWidget,
    QLabel,
    QLineEdit,
    QPushButton,
    QVBoxLayout,
    QMessageBox,
)

from db.user_service import login_user
from gui.gui import NetworkMonitorGUI

```



```

class LoginWindow(QWidget):

    def __init__(self):
        super().__init__()

        self.setWindowTitle("Login")
        self.setFixedSize(500, 400)

        layout = QVBoxLayout()

        self.username = QLineEdit()
        self.username.setPlaceholderText("Username")

        self.password = QLineEdit()
        self.password.setPlaceholderText("Password")
        self.password.setEchoMode(QLineEdit.Password)

        login_btn = QPushButton("Login")
        signup_btn = QPushButton("Signup")

        login_btn.clicked.connect(self.handle_login)
        signup_btn.clicked.connect(self.open_signup)

        layout.addWidget(QLabel("Network Monitor Login"))
        layout.addWidget(self.username)
        layout.addWidget(self.password)
        layout.addWidget(login_btn)
        layout.addWidget(signup_btn)

        self.setLayout(layout)

    def handle_login(self):
        username = self.username.text()
        password = self.password.text()

        success, result = login_user(username, password)

```

```

        if success:
            QMessageBox.information(self, "Success", "Login s
uccessful")

            self.monitor_window = NetworkMonitorGUI()
            self.monitor_window.show()

            self.close()

        else:
            QMessageBox.warning(self, "Error", result)

    def open_signup(self):
        from signup_window import SignupWindow

        self.signup_window = SignupWindow()
        self.signup_window.show()

```

gui/node_availability_chart.py

```

from PySide6.QtCharts import QChart, QChartView, QPieSeries
from PySide6.QtGui import QPainter, QColor
from PySide6.QtCore import Qt

class NodeAvailabilityChart(QChartView):
    def __init__(self, parent=None):

        super().__init__(parent)

        self.chart_obj = QChart()
        self.chart_obj.setTitle("Node Availability Distributi
on")

        self.series = QPieSeries()

```

```

self.series.setPieSize(0.7)

self.chart_obj.addSeries(self.series)

self.chart_obj.legend().setVisible(True)
self.chart_obj.legend().setAlignment(Qt.AlignRight)

self.setChart(self.chart_obj)
self.setRenderHint(QPainter.Antialiasing)

def update_data(self, up, degraded, down):

    self.series.clear()

    total = up + degraded + down

    if total == 0:
        return

    # calculate percentages
    up_pct = (up / total) * 100
    degraded_pct = (degraded / total) * 100
    down_pct = (down / total) * 100

    # create slices with percentage labels
    up_slice = self.series.append(
        f"UP ({up_pct:.1f}%)", up
    )

    degraded_slice = self.series.append(
        f"DEGRADED ({degraded_pct:.1f}%)", degraded
    )

    down_slice = self.series.append(
        f"DOWN ({down_pct:.1f}%)", down
    )

```

```

# colors
up_slice.setBrush(QColor("#2ecc71"))
degraded_slice.setBrush(QColor("#f1c40f"))
down_slice.setBrush(QColor("#e74c3c"))

# show labels
up_slice.setLabelVisible(True)
degraded_slice.setLabelVisible(True)
down_slice.setLabelVisible(True)

# explode largest slice
slices = [up_slice, degraded_slice, down_slice]

largest = max(slices, key=lambda s: s.value())

largest.setExploded(True)
largest.setExplodeDistanceFactor(0.1)

self.chart_obj.update()

```

gui/node_pie_chart.py

```

# gui/node_pie_chart.py
from PySide6.QtCharts import QChart, QChartView, QPieSeries, QPieSlice
from PySide6.QtCore import Qt
from PySide6.QtGui import QColor, QPainter

class NodePieChart(QChartView):
    def __init__(self, node_name, parent=None):
        super().__init__(parent)
        self.node_name = node_name

        self.chart_obj = QChart()

```

```

self.chart_obj.setTitle(f"{node_name} Status")

self.series = QPieSeries()
self.chart_obj.addSeries(self.series)

self.setChart(self.chart_obj)
self.setRenderHint(QPainter.Antialiasing)

def update_data(self, availability_percent, packet_loss_p
ercent):
    """
    Update pie chart with availability and packet loss pe
rcentages
    availability_percent: 0-100
    packet_loss_percent: 0-100
    """
    self.series.clear()

    # Create slices
    availability_slice = QPieSlice(
        f"Available {availability_percent}%", availabilit
y_percent
    )
    availability_slice.setColor(QColor("#2ecc71")) # gre
en

    loss_slice = QPieSlice(f"Loss {packet_loss_percen
t}%", packet_loss_percent)
    loss_slice.setColor(QColor("#e74c3c")) # red

    self.series.append(availability_slice)
    self.series.append(loss_slice)

    # Label styling

```

```
availability_slice.setLabelVisible(True)
loss_slice.setLabelVisible(True)
```

gui/signup_window.py

```
import sys
import os

# Add parent directory to path
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from PySide6.QtWidgets import (
    QWidget,
    QLabel,
    QLineEdit,
    QPushButton,
    QVBoxLayout,
    QMessageBox,
)

from db.user_service import create_user

class SignupWindow(QWidget):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Signup")
        self.setFixedSize(300, 250)

        layout = QVBoxLayout()

        self.username = QLineEdit()
        self.username.setPlaceholderText("Username")
```

```

self.email = QLineEdit()
self.email.setPlaceholderText("Email")

self.password = QLineEdit()
self.password.setPlaceholderText("Password")
self.password.setEchoMode(QLineEdit.Password)

signup_btn = QPushButton("Create Account")
signup_btn.clicked.connect(self.handle_signup)

layout.addWidget(QLabel("Signup"))
layout.addWidget(self.username)
layout.addWidget(self.email)
layout.addWidget(self.password)
layout.addWidget(signup_btn)

self.setLayout(layout)

def handle_signup(self):

    success, message = create_user(
        self.username.text(), self.email.text(), self.password.text()
    )

    if success:
        QMessageBox.information(self, "Success", message)
        self.close()
    else:
        QMessageBox.warning(self, "Error", message)

```

gui/summary.py

```

# gui/summary.py

from PySide6.QtWidgets import QWidget, QLabel, QHBoxLayout

```

```

class HealthSummaryWidget(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)

        layout = QHBoxLayout()
        self.setLayout(layout)

        self.label = QLabel("Loading summary...")
        self.label.setStyleSheet("""
            font-size: 16px;
            font-weight: bold;
            padding: 8px;
        """)

        layout.addWidget(self.label)

    def update_summary(self, states: dict):
        if not states:
            self.label.setText("No nodes available")
            return

        total = len(states)
        up = 0
        degraded = 0
        down = 0
        latency_sum = 0
        latency_count = 0

        for node, info in states.items():
            state = info["state"]

            if state == "UP":
                up += 1
            elif state == "DEGRADED":

```



```

        degraded += 1
    elif state == "DOWN":
        down += 1

    try:
        latency = float(info["latency"])
        latency_sum += latency
        latency_count += 1
    except:
        pass

    avg_latency = latency_sum / latency_count if latency_
count > 0 else 0

    text = (
        f"Nodes: {total}      "
        f"UP: {up}          "
        f"DEGRADED: {degraded}    "
        f"DOWN: {down}         "
        f"Avg Latency: {avg_latency:.1f} ms"
    )

    self.label.setText(text)

```

gui/widgets.py

```

# sidebar, panels

# gui/widgets.py
from PySide6.QtWidgets import QListWidget, QListWidgetItem, Q
MessageBox
from PySide6.QtGui import QBrush, QColor

class NodeListWidget(QListWidget):

```

```

def __init__(self, parent=None):
    super().__init__(parent)
    self.setFixedWidth(200)
    self.previous_states = {}

def refresh(self, states, parent_window):
    self.clear()

    for node, info in states.items():
        state = info["state"]

        # choose color
        if state == "UP":
            color = QColor("#2ecc71") # green
        elif state == "DEGRADED":
            color = QColor("#f1c40f") # yellow
        else:
            color = QColor("#e74c3c") # red

        item = QListWidgetItem(f"● {node} {state}")
        item.setForeground(QBrush(color))
        self.addItem(item)

        # alert on transition → DOWN
        prev = self.previous_states.get(node)
        if state == "DOWN" and prev != "DOWN":
            QMessageBox.warning(
                parent_window,
                "Node DOWN Alert",
                f"Node: {node}\nIP: {info['ip']}\nStatus:
DOWN",
            )

        self.previous_states[node] = state

```

logs/log.txt

```
2026-02-19 12:41:47 | node_1_phone | 192.168.1.39 | UP | type
= wifi | latency=6.547 | fails=0
2026-02-19 12:41:47 | node_2_laptop | 192.168.1.34 | UP | typ
e = wifi | latency=0.16 | fails=0
2026-02-19 12:41:47 | node_3_router | 192.168.1.1 | UP | type
= router | latency=25.422 | fails=0
2026-02-19 12:41:52 | node_1_phone | 192.168.1.39 | DEGRADED
| type = wifi | latency=110.509 | fails=0
2026-02-19 12:41:52 | node_2_laptop | 192.168.1.34 | UP | typ
e = wifi | latency=0.176 | fails=0
2026-02-19 12:41:53 | node_3_router | 192.168.1.1 | UP | type
= router | latency=4.839 | fails=0
2026-02-19 12:41:58 | node_1_phone | 192.168.1.39 | UP | type
= wifi | latency=8.129 | fails=0
```

src/config.py

```
# nodes & constants
LATENCY_THRESHOLD = 100 # ms

nodes = {
    "node_1_phone": {
        "ip": "192.168.1.39",
        "state": "UP",
        "network_type": "wifi",
        "fail_count": 0,
        "last_latency": None,
    },
    "node_2_laptop": {
        "ip": "192.168.1.34",
        "state": "UP",
```

```

        "network_type": "wifi",
        "fail_count": 0,
        "last_latency": None,
    },
    "node_3_router": {
        "ip": "192.168.1.1",
        "state": "UP",
        "network_type": "router",
        "fail_count": 0,
        "last_latency": None,
    },
}

```

src/main.py

```

import time
import threading
import sys
import os

# Add parent directory to path to access gui folder
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from config import nodes
from monitor import run_monitor
from gui.gui import NetworkMonitorGUI
from gui.login_window import LoginWindow

from PySide6.QtWidgets import QApplication

def monitor_loop():
    while True:
        run_monitor(nodes)

```

```

        time.sleep(5)

if __name__ == "__main__":
    print("Starting system...")

    app = QApplication(sys.argv)

    # Show login window first
    login_window = LoginWindow()
    login_window.show()

    monitor_thread = threading.Thread(target=monitor_loop, da
emon=True)
    monitor_thread.start()

    sys.exit(app.exec())

```

src/moniter.py

```

import os
from ping import ping_node
from config import LATENCY_THRESHOLD
from datetime import datetime
from db.log_service import insert_log

# optional file logger
try:
    from db.file_log_service import write_log_file
except Exception:
    write_log_file = None

def evaluate_status(ping_result: dict, previous_state: str, f
ail_count: int) -> str:

```

```

    if not ping_result["success"]:
        if fail_count >= 3:
            return "DOWN"
        return previous_state

    if ping_result["latency"] is None:
        return "DEGRADED"

    if ping_result["latency"] > LATENCY_THRESHOLD:
        return "DEGRADED"

    return "UP"

def run_monitor(nodes: dict):

    for node_name, node in nodes.items():
        result = ping_node(node["ip"])

        if result["success"]:
            node["fail_count"] = 0
        else:
            node["fail_count"] += 1

        node["state"] = evaluate_status(result, node["state"], node["fail_count"])

        node["last_latency"] = result["latency"]

        print(f"{node_name} → {node['state']} | latency={node['last_latency']}")

        time_stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        # write to file logger if available, otherwise write

```

```

raw_line
    if write_log_file:
        try:
            write_log_file(
                node_name,
                node["ip"],
                node["network_type"],
                node["state"],
                node["last_latency"],
                node["fail_count"],
            )
        except Exception:
            # fallback to raw write
            log_line = (
                f"{time_stamp} | {node_name} | {node['i
p']} | "

                f"{node['state']} | "
                f"type = {node['network_type']} | "
                f"latency={node['last_latency']} | "
                f"fails={node['fail_count']}"
            )

            write_log(log_line)
        else:
            log_line = (
                f"{time_stamp} | {node_name} | {node['ip']} | "

                f"{node['state']} | "
                f"type = {node['network_type']} | "
                f"latency={node['last_latency']} | "
                f"fails={node['fail_count']}"
            )

            write_log(log_line)

# INSERT INTO MONGODB (FIXED POSITION)

```

```

insert_log(
    node_name,
    node["ip"],
    node["network_type"],
    node["state"],
    node["last_latency"],
    node["fail_count"],
)

```

```

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__
file__)))
LOG_FILE = os.path.join(BASE_DIR, "logs", "log.txt")

```

```

def write_log(message: str):
    with open(LOG_FILE, "a") as f:
        f.write(message + "\n")

```

src/ping.py

```

# ping logic
import subprocess

def ping_node(ip: str) -> dict:
    result = subprocess.run(
        ["ping", "-c", "1", ip],
        capture_output=True,
        text=True
    )

    if result.returncode != 0:
        return {
            "success": False,
            "latency": None,
            "error": "ping_failed"

```



```

    }

    latency = None
    for line in result.stdout.splitlines():
        if "time=" in line:
            time_part = line.split("time=")[1]
            latency = float(time_part.split(" ")[0])
            break

    if latency is None:
        return {
            "success": True,
            "latency": None,
            "error": "latency_parse_failed"
        }

    return {
        "success": True,
        "latency": latency,
        "error": None
    }

```

src/run_monitor.py

```

#!/usr/bin/env python3
# run_monitor.py – run the monitor loop in foreground (no GU
I)
import time
import sys
import os

# ensure project src is on path
sys.path.insert(0, os.path.dirname(os.path.abspath(__file_
_)))

from monitor import run_monitor

```

```

from config import nodes

def main():
    print("Starting monitor loop (foreground). Ctrl+C to stop.")
    try:
        while True:
            run_monitor(nodes)
            time.sleep(5)
    except KeyboardInterrupt:
        print("Monitor stopped")

if __name__ == "__main__":
    main()

```

src/run_gui.py

```

#!/usr/bin/env python3
# run_gui.py – start the GUI (login + monitor window) without
starting monitor thread
import sys
import os

# ensure project src is on path
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from PySide6.QtWidgets import QApplication
from gui.login_window import LoginWindow

def main():
    app = QApplication(sys.argv)

```

```

login = LoginWindow()
login.show()

sys.exit(app.exec())

if __name__ == "__main__":
    main()

```

src/db/file_log_service.py

```

import os
from datetime import datetime

BASE_DIR = os.path.dirname(os.path.dirname(os.path.dirname(__
file__)))

LOG_FILE = os.path.join(BASE_DIR, "logs", "log.txt")

def write_log_file(node_name, ip, network_type, state, latenc
y, fail_count):

    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    log_line = (
        f"{timestamp} | {node_name} | {ip} | "
        f"{state} | type = {network_type} | "
        f"latency={latency} | fails={fail_count}"
    )

    # ensure directory exists
    log_dir = os.path.dirname(LOG_FILE)

```

```

if not os.path.exists(log_dir):
    try:
        os.makedirs(log_dir, exist_ok=True)
    except Exception:
        pass

with open(LOG_FILE, "a") as f:
    f.write(log_line + "\n")

```

src/db/log_service.py

```

from db.mongo import logs_collection
from datetime import datetime, timedelta

def get_logs_by_range(range_type="all"):
    """
    range_type:
    all
    day
    week
    month
    """
    if logs_collection is None:
        return []

    query = {}

    now = datetime.now()

    if range_type == "day":
        query["timestamp"] = {"$gte": now - timedelta(days=
1)}}
    elif range_type == "week":
        query["timestamp"] = {"$gte": now - timedelta(weeks=
1)}}

```

```

        elif range_type == "month":
            query["timestamp"] = {"$gte": now - timedelta(days=30)}

        logs = list(logs_collection.find(query).sort("timestamp", -1).limit(200))

        return logs

def insert_log(node_name, ip, network_type, state, latency, fails):
    if logs_collection is None:
        # MongoDB not configured/available; skip insert silently
        return

    try:
        logs_collection.insert_one(
            {
                "timestamp": datetime.now(),
                "node_name": node_name,
                "ip": ip,
                "network_type": network_type,
                "state": state,
                "latency": latency,
                "fails": fails,
            }
        )
    except Exception as e:
        print(f"[ERROR] Failed to insert log for {node_name}: {str(e)[:100]}")

```

src/db/mongo.py

```

# NEW code with ERROR HANDLING
try:
    import pymongo
except Exception:
    pymongo = None

import os

try:
    from dotenv import load_dotenv

    load_dotenv()
except Exception:
    # dotenv not available; proceed without loading .env
    pass

mongo_uri = os.getenv("MONGO_URI")

client = None
db = None
logs_collection = None
users_collection = None

if pymongo is None:
    print("pymongo not installed; MongoDB features disabled")
else:
    try:
        if not mongo_uri:
            raise ValueError("MONGO_URI not found in environm
ent")

        client = pymongo.MongoClient(mongo_uri, serverSelecti
onTimeoutMS=5000)

```

```

# Test connection
client.admin.command("ping")

db = client["network_monitor"]

logs_collection = db["logs"]
users_collection = db["users"]

print("✅ MongoDB connected successfully")

except Exception as e:
    print(f"❌ MongoDB connection unavailable: {e}")

```

user_service.py

```

from db.mongo import users_collection
import hashlib

def hash_password(password):
    """Hash password using SHA256"""
    return hashlib.sha256(password.encode()).hexdigest()

def login_user(username, password):
    """
    Authenticate user with username and password
    Returns: (success: bool, message: str)
    """
    try:
        if not username or not password:
            return False, "Username and password are required"

        user = users_collection.find_one({"username": username})

```

```

e}))

    if not user:
        return False, "User not found"

    # Check password
    hashed_password = hash_password(password)
    if user["password"] == hashed_password:
        return True, "Login successful"
    else:
        return False, "Incorrect password"

except Exception as e:
    return False, f"Login error: {str(e)[:100]}"

def create_user(username, email, password):
    """
    Create a new user account
    Returns: (success: bool, message: str)
    """
    try:
        if not username or not email or not password:
            return False, "All fields are required"

        # Check if user already exists
        existing_user = users_collection.find_one({"username": username})
        if existing_user:
            return False, "Username already exists"

        # Check if email already exists
        existing_email = users_collection.find_one({"email": email})
        if existing_email:
            return False, "Email already exists"

```



```
# Hash password
hashed_password = hash_password(password)

# Create user
users_collection.insert_one(
    {
        "username": username,
        "email": email,
        "password": hashed_password,
    }
)

return True, "Account created successfully! You can now login."

except Exception as e:
    return False, f"Signup error: {str(e)[:100]}"
```